

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №4
«Технический дизайн микросервисной архитектуры»

Выполнил:

Олейник Полина

Группа 2.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Разделить монолитное бэкенд-приложение на микросервисы и спроектировать микросервисную архитектуру. Для этого необходимо:

- разделить монолитное приложение на микросервисы, придерживаясь концепции database-per-service (у каждого сервиса своя база данных);
- спроектировать микросервисную архитектуру: отразить связи между сервисами и описать способы их взаимодействия;
- спроектировать разделение базы данных на обособленные базы;
- спроектировать и описать эндпоинты, необходимые для межсервисного взаимодействия, в формате спецификации OpenAPI;
- описать варианты запросов и ответов и задокументировать возможные ошибки;
- сформировать документ с конкретными требованиями и шагами разделения монолита на отдельные микросервисы.

Тема проекта: вариант 1 – приложение для бронирования столиков в ресторанах. Исходное монолитное приложение реализовано в ЛР1.

Ход работы

1. Исходное состояние и требования к разделению

1.1 Исходное приложение

Исходное приложение из ЛР1 – монолит на Express и TypeORM с одной базой PostgreSQL: 11 таблиц и 3 перечисления, 13 контроллеров, 4 сервиса бизнес-логики, 4 middleware, 43 операции публичного API. Все части приложения свободно обращаются к любым таблицам, а внешние ключи связывают таблицы напрямую.

1.2 Что мешает разделению

При выборе границ сервисов были найдены места, где части монолита тесно связаны между собой. Они приведены в таблице 1 вместе с решением для каждого.

Таблица 1 – Места связи частей монолита

Место связи	Как устроено в монолите	Решение при разделении
Бронь и три внешних ключа	Booking ссылается на User, RestaurantTable и TimeSlot	В разных сервисах внешние ключи невозможны. Остаются логические ссылки (id), а существование столика и слота проверяется запросом к restaurants
Отзывы и назначения администраторов	Review и RestaurantAdmin ссылаются на User	Ссылка user_id без внешнего ключа; имена авторов запрашиваются у auth одним запросом на страницу
Свободные столики	LEFT JOIN столиков с бронями в одной базе	Столики берутся у restaurants, занятость из базы bookings, в коде из первых вычитаются вторые
История броней	BookingDetails содержит ресторан, столик и слот	bookings запрашивает эти данные у restaurants одним запросом на страницу
Права администратора ресторана	Таблица RestaurantAdmin	Таблица принадлежит restaurants; bookings запрашивает у него список ресторанов пользователя
Смена роли при назначении администратора	Поле User.role	restaurants обращается к auth, который владеет пользователями и ролями
ON DELETE RESTRICT для истории	Бронь запрещает удалить столик, слот и ресторан	Перед удалением restaurants спрашивает у bookings, есть ли брони
Рейтинг ресторана	Пересчёт в транзакции вместе с отзывом	Отзывы остаются в restaurants, транзакция остаётся локальной

1.3 Требования к разделению

- публичный API из Д32 (43 операции, адреса, тела запросов и коды ответов) остаётся прежним, поэтому клиенты и коллекции Postman не меняются;
- каждый сервис владеет своей базой данных и не обращается к чужой: данные другого сервиса запрашиваются только через его API;
- сервисы общаются по HTTP через отдельный внутренний API, который недоступен снаружи;
- токен пользователя (JWT) каждый сервис проверяет самостоятельно, без обращения к auth;
- ограничения целостности из Д31 сохраняются: те, что нельзя автоматически проверять в базе из-за границ сервисов, проверяется с помощью запросов по внутреннему API;

- недоступность одного сервиса не должна приводить к неопределенному поведению: ошибки и таймауты обрабатываются единообразно (раздел 7).

2. Разделение на сервисы

2.1 Границы сервисов

Границы проведены по владению данными и ответственности: сервис владеет своими таблицами и всей логикой, которая с ними связана. Список сервисов приведён в таблице 2.

Таблица 2 – Микросервисы

Сервис	Порт	База данных	Ответственность	Операций
auth-service	3001	auth_db	Пользователи и роли, регистрация, вход, выдача JWT, профиль пользователя	4
restaurants-service	3002	restaurants_db	Каталог: рестораны, справочники городов и кухонь, фото, меню, столики, слоты, отзывы, администраторы ресторанов	33
bookings-service	3003	bookings_db	Бронирования: создание, история, смена статуса, поиск свободных столиков	6

Ключевые решения по границам:

- **Отзывы находятся в restaurants-service.** Рейтинг ресторана пересчитывается из отзывов в одной транзакции, а при выделении отзывов в отдельный сервис пришлось бы синхронизировать рейтинг событиями.
- **Администраторы ресторана тоже в restaurants-service.** Право управлять рестораном хранится рядом с рестораном, а другие сервисы узнают его по запросу.
- **Свободные столики считает bookings-service.** Он владеет данными о занятости, а список столиков запрашивает у restaurants.
- **Пользователи и роли только в auth-service.** Остальные сервисы получают роль и идентификатор из токена и не хранят данные пользователей.

2.2 Распределение публичных операций

Все 43 операции Д32 распределены между сервисами (таблица 3); ни одна не потеряна и не дублируется.

Таблица 3 – Распределение публичных операций

Сервис	Операции	Число
auth-service	POST /auth/register, POST /auth/login, GET и PATCH /users/me	4
restaurants-service	справочники городов и кухонь (6), рестораны (5), фото (3), меню (4), столики (4), слоты (4), отзывы (4), администраторы (3)	33
bookings-service	POST и GET /bookings, GET /bookings/{id}, PATCH /bookings/{id}/status, GET /users/me/bookings, GET /restaurants/{id}/availability	6
Итого	Публичный API из Д32 не меняется	43

3. Архитектура и способы взаимодействия

Архитектура показана на рисунке 1. Клиент обращается к единой точке входа (gateway), которая по адресу запроса направляет его в нужный сервис. Сервисы общаются друг с другом напрямую по внутреннему API; у каждого сервиса своя база.

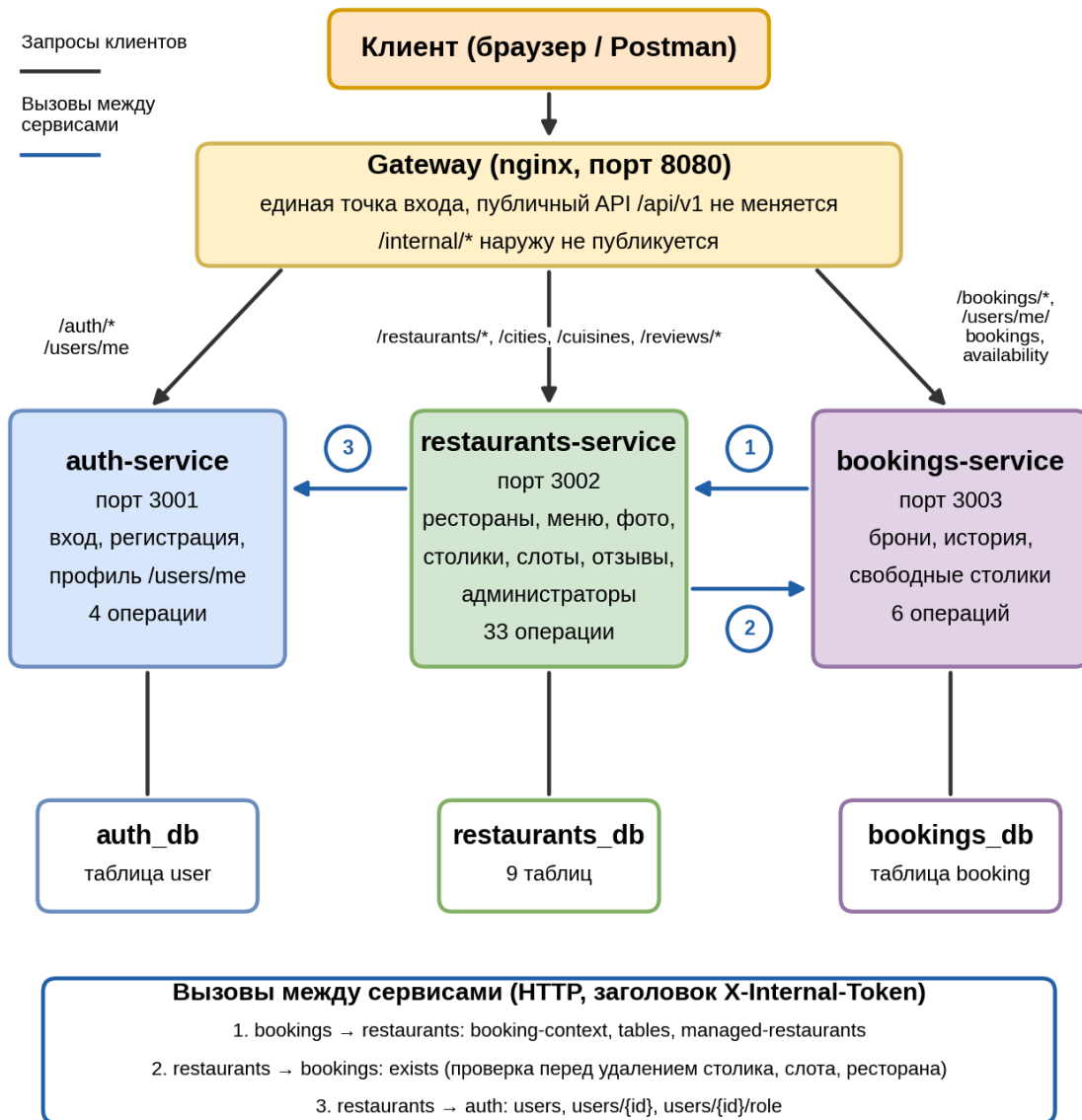


Рисунок 1 – Архитектура: gateway, три сервиса, их базы данных и вызовы между сервисами

3.1 Способы взаимодействия

- **Клиент – сервис:** через gateway (nginx), по публичному API /api/v1, формат JSON.
- **Сервис – сервис:** синхронные HTTP-запросы к внутреннему API (адреса /internal/*), формат JSON. Синхронный вызов выбран потому, что результат нужен сразу, например для проверки столика при создании брони.
- **Аутентификация пользователя:** auth-service выдаёт JWT, подписанный общим секретом JWT_SECRET_KEY. Остальные

сервисы проверяют подпись самостоятельно и берут из токена идентификатор и роль. Вызов auth при каждом запросе не нужен.

- **Аутентификация сервиса:** каждый внутренний запрос содержит заголовок X-Internal-Token с общим ключом INTERNAL_API_KEY. Gateway не публикует адреса /internal/*, поэтому клиент до них добраться не может.
- **Таймауты:** запросы между сервисами ограничены 3 секундами.
- **Очереди сообщений** (RabbitMQ или Kafka) в эту архитектуру добавляются позже, в Д35, для событий, которым не нужен ответ сразу.

3.2 Настройки сервисов

У всех сервисов одинаковые JWT_SECRET_KEY и INTERNAL_API_KEY. У каждого свои APP_PORT и параметры подключения к своей базе, а также адреса других сервисов: AUTH_SERVICE_URL, RESTAURANTS_SERVICE_URL, BOOKINGS_SERVICE_URL.

3.3 Маршрутизация на gateway

Правила gateway приведены в таблице 4. Адрес /restaurants/{id}/availability начинается с /restaurants, но обслуживается сервисом бронирований, а /users/me/bookings начинается с /users/me, но тоже уходит в него. Поэтому правила упорядочены от более точных путей к общим, а публичный API остаётся таким же, как в Д32.

Таблица 4 – Маршрутизация на gateway

Путь (после /api/v1)	Куда направляется	Примечание
/auth/*, /users/me	auth-service	Точное совпадение /users/me
/users/me/bookings	bookings-service	Проверяется раньше /users/me, поэтому идёт в другой сервис
/bookings, /bookings/*	bookings-service	
/restaurants/{id}/availability	bookings-service	Более специфичный путь проверяется раньше общего /restaurants/*
/cities, /cuisines, /restaurants, /restaurants/*, /reviews/*	restaurants-service	
/internal/*	не публикуется	Внутренний API доступен только сервисам во внутренней сети

3.4 Кто кого вызывает

Направления вызовов между сервисами приведены в таблице 5. Между restaurants и bookings есть взаимная зависимость: bookings нужны данные о столиках, а restaurants перед удалением нужно знать о бронях. Но опасности взаимного ожидания (deadlock) нету. У обоих микросервисов внутренние эндпоинты отвечают только из своей базы и никого больше не вызывают:

- booking-context в restaurants читает свою базу и сразу отвечает;
- exists в bookings тоже читает свою базу и сразу отвечает.

То есть цикл не создаёт блокировок, но связывает сервисы сильнее, чем хотелось бы. Способы ослабить эту связь описаны в разделе 7.

Таблица 5 – Вызовы между сервисами

Направление	Внутренние эндпоинты	Зачем
bookings → restaurants	GET /internal/booking-context, GET /internal/restaurants/{id}/tables, GET /internal/users/{id}/managed-restaurants	Создание брони, история броней, свободные столики, права администратора ресторана
restaurants → bookings	GET /internal/bookings/exists	Проверка перед удалением столика, слота или ресторана
restaurants → auth	GET /internal/users, GET /internal/users/{id}, PATCH /internal/users/{id}/role	Авторы отзывов, список администраторов, назначение администратора ресторана
auth → остальные	–	Не вызывает никого: токен другие сервисы проверяют сами

4. Разделение базы данных

Монолитная база разделена на три независимые базы (таблица 6). Физически они могут находиться на одном сервере PostgreSQL, но у каждой свой пользователь и пароль, и сервис подключается только к своей базе.

Таблица 6 – Разделение базы данных

База данных	Таблицы	Перечисление
auth_db	user	Role
restaurants_db	city, cuisine, restaurant, restaurant_admin, restaurant_photo, menu_item, restaurant_table, time_slot, review	MenuCategory
bookings_db	booking	BookingStatus

Изменения схемы по сравнению с диаграммой ДЗ1 приведены в таблице 7.

Таблица 7 – Изменения схемы при разделении

Что	Изменение	Как обеспечивается целостность
restaurant_admin.user_id	Внешний ключ на user удалён, остаётся обычное поле	Существование пользователя проверяется через auth при назначении администратора
review.user_id	То же	Автор известен из токена; составное ограничение UNIQUE (user_id, restaurant_id) остаётся
booking.user_id, restaurant_table_id, time_slot_id	Внешние ключи удалены	Столик и слот проверяются через restaurants при создании; частичный уникальный индекс по (столик, слот, дата) остаётся в bookings_db
booking.restaurant_id	Новое поле: копия идентификатора ресторана на момент создания брони	Нужно для фильтра «брони ресторана» и проверки прав без запроса к restaurants на каждую бронь
ON DELETE RESTRICT (столик, слот, ресторан ← бронь)	Средствами БД больше не проверяется	Заменён вызовом bookings /internal/bookings/exists перед удалением
Остальные ограничения	Без изменений	UNIQUE, CHECK, CASCADE и RESTRICT внутри restaurants_db, CHECK на guests_count в bookings_db

5. Внутренние эндпоинты (OpenAPI)

Для межсервисного взаимодействия спроектировано 7 внутренних эндпоинтов (таблица 8). Они описаны в файле `openapi-internal.yaml` (OpenAPI 3.0.3): 7 операций, 11 схем; у каждого пути в поле `servers` указан сервис, который на него отвечает. Все операции защищены схемой `internalToken` (заголовок `X-Internal-Token`).

Таблица 8 – Внутренние эндпоинты

№	Метод и адрес	Отвечает	Вызывает	Назначение
1	GET /internal/users?ids=	auth	restaurants	Краткие данные пользователей пачкой (авторы отзывов, администраторы)
2	GET /internal/users/{userId}	auth	restaurants	Существование пользователя и его роль
3	PATCH /internal/users/{userId}/role	auth	restaurants	USER → RESTAURANT_ADMIN при назначении администратора

№	Метод и адрес	Отвечает	Вызывает	Назначение
4	GET /internal/booking-context?table_ids=&time_slot_ids=	restaurants	bookings	Столики, слоты и рестораны для проверки и отображения броней
5	GET /internal/restaurants/{restaurantId}/tables	restaurants	bookings	Столики ресторана для расчёта свободных
6	GET /internal/users/{userId}/managed-restaurants	restaurants	bookings	Рестораны, которыми управляет пользователь
7	GET /internal/bookings/exists?table_ids=&time_slot_ids=	bookings	restaurants	Есть ли брони на столики или слоты

Примеры запросов и ответов.

Данные для проверки брони. Запрос к restaurants-service:

```
GET /internal/booking-context?table_ids=3&time_slot_ids=2
X-Internal-Token: <INTERNAL_API_KEY>
```

Ответ 200:

```
{
  "tables": [
    { "id": 3, "restaurant_id": 5, "number": "A1", "capacity": 4,
      "zone": "Теппака", "is_active": true }
  ],
  "time_slots": [
    { "id": 2, "restaurant_id": 5, "start_time": "14:00", "end_time": "16:00",
      "is_active": true }
  ],
  "restaurants": [
    { "id": 5, "name": "Траттория Порто", "address": "Невский проспект, 10" }
  ]
}
```

Проверка наличия броней перед удалением столика. Запрос к bookings-service GET /internal/bookings/exists?table_ids=3, ответ 200:

```
{ "exists": true, "count": 2 }
```

В этом случае restaurants-service клиенту возвращает 409 в формате публичного API:

```
{ "statusCode": 409, "message": "Нельзя удалить столик: на него есть бронирования" }
```

Смена роли при назначении администратора. Запрос к auth-service PATCH /internal/users/7/role:

```
{ "role": "RESTAURANT_ADMIN" }
```

Ответ 200:

```
{ "id": 7, "role": "RESTAURANT_ADMIN", "first_name": "Иван", "last_name": "Ресторанов" }
```

Ошибки внутреннего API и реакция вызывающего сервиса приведены в таблице 9. Формат ошибки тот же, что у публичного API: {statusCode, message, errors?}.

Таблица 9 – Ошибки внутреннего API

Код	Когда возвращается	Что делает вызывающий сервис
400	Неверный список идентификаторов; не передан ни один обязательный параметр	Это ошибка вызывающего кода: клиенту возвращается 500, подробности пишутся в журнал
401	Заголовок X-Internal-Token не передан или неверен	Ошибка настройки: клиенту 500, запись в журнал
404	Пользователь или ресторан не найден	По смыслу операции: 404 клиенту (например, назначение несуществующего пользователя)
5xx или нет соединения	Вызываемый сервис не запущен или упал	Клиенту 503; операция не выполняется или результат неполный (см. раздел 7)
Таймаут 3 с	Вызываемый сервис не ответил вовремя	Клиенту 504

6. Сценарии взаимодействия

6.1 Создание брони

Последовательность вызовов показана на рисунке 2. Клиент обращается к bookings-service, который проверяет токен и тело запроса, затем запрашивает у restaurants-service данные о столике и слоте и только после успешных проверок записывает бронь в свою базу. Защиту от двойной брони обеспечивают две проверки: запрос занятости и частичный уникальный индекс в bookings_db.

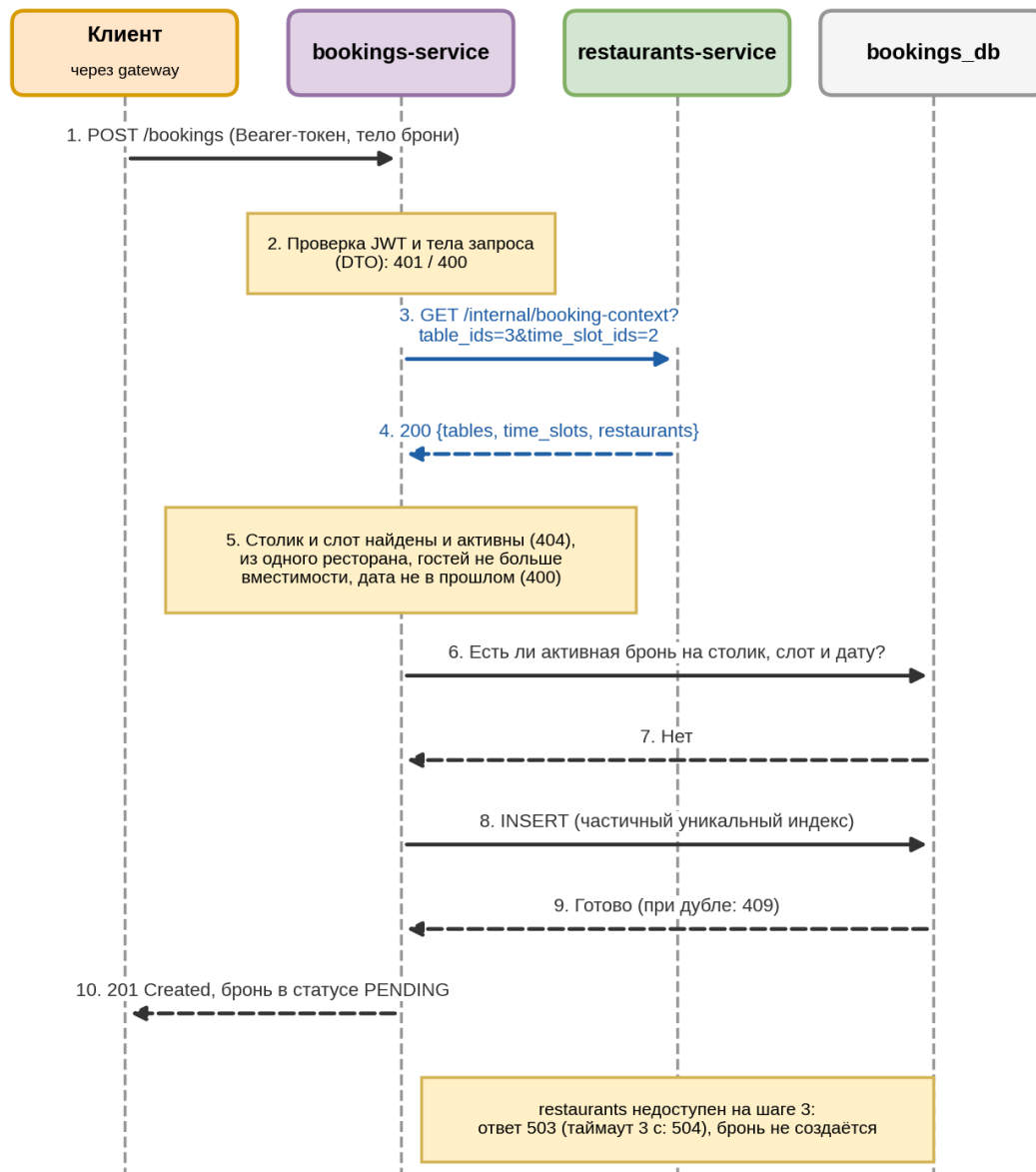


Рисунок 2 – Создание брони: вызовы между клиентом, сервисами и базой

6.2 Остальные сценарии

- **Свободные столики** (GET /restaurants/{id}/availability). bookings-service запрашивает у restaurants данные слота (эндпоинт 4) и активные столики нужной вместимости (эндпоинт 5), затем в своей базе находит столики с активной бронью на выбранные дату и слот и исключает их. В монолите это делал один запрос с LEFT JOIN, теперь это вычитание двух списков в коде.
- **История броней** (GET /users/me/bookings). bookings-service выбирает страницу броней пользователя из своей базы и одним

запросом к restaurants (эндпоинт 4) получает данные ресторанов, столиков и слотов для всей страницы.

- **Список броней для администратора** (GET /bookings). Для администратора ресторана bookings-service запрашивает список его ресторанов (эндпоинт 6) и ограничивает выборку по полю restaurant_id; для роли ADMIN запрос не нужен. Это поле и добавлено в таблицу booking.
- **Назначение администратора** (POST /restaurants/{id}/admins). restaurants-service проверяет пользователя (эндпоинт 2, при отсутствии 404), при роли USER меняет её на RESTAURANT_ADMIN (эндпоинт 3) и записывает назначение в свою таблицу (при повторе 409).
- **Удаление столика, слота, ресторана.** restaurants-service спрашивает bookings (эндпоинт 7); если брони есть, возвращает 409, иначе удаляет данные в своей базе.
- **Отзывы.** Создание отзыва и пересчёт рейтинга остаются локальной транзакцией restaurants-service. При выводе списка имена авторов запрашиваются у auth пачкой (эндпоинт 1).

7. Сбои и согласованность

Так как данные теперь находятся в разных сервисах, сбои и рассинхронизация возможны. Принятые решения приведены в таблице 10.

Таблица 10 – Сбои и согласованность

Ситуация	Поведение	Почему так
restaurants недоступен при создании брони	Ответ 503, бронь не создаётся	Без данных о столике и слоте бронь проверить нельзя
restaurants недоступен при выводе истории броней	Ответ 200: брони возвращаются, вложенные объекты restaurant, restaurant_table и time_slot равны null	История важнее подробностей; частичный ответ лучше ошибки
auth недоступен при выводе отзывов или администраторов	Ответ 200, поле user равно null	Отзывы и список администраторов остаются читаемыми
bookings недоступен при удалении столика, слота или ресторана	Ответ 503, удаление не выполняется	Нельзя удалять данные, не зная, есть ли на них брони
Сбой между шагами назначения администратора	Сначала выполняется смена роли (идемпотентна), затем запись в restaurant_admin; при сбое клиент повторяет запрос	Роль без назначенных ресторанов ничего не даёт: права проверяются по таблице restaurant_admin

Ситуация	Поведение	Почему так
Столик отключён или удалён между проверкой и вставкой брони	Допускается: узкое окно гонки	От двойной брони защищает частичный уникальный индекс в <code>bookings_db</code> ; остальное разрешается отменой брони администратором
Повторы запросов	GET-запросы повторяются один раз, PATCH роли идемпотентен, POST автоматически не повторяются	Повтор небезопасных операций мог бы создать дубликаты

Как ослабить взаимную зависимость `restaurants` и `bookings`. Возможны два пути: перейти на события (`bookings` публикует «бронь создана» и «бронь отменена», `restaurants` хранит признак «на столик есть брони»; это можно сделать после ДЗ5) или запретить физическое удаление столиков, слотов и ресторанов и оставить только отключение (`is_active = false`), тогда вызов `exists` не понадобится.

8. План разделения (шаги для ЛР2)

1. Создать структуру репозитория: `services/auth`, `services/restaurants`, `services/bookings` и `gateway`. У каждого сервиса свой `package.json`, `tsconfig.json` и `.env` (общие `JWT_SECRET_KEY` и `INTERNAL_API_KEY`, свои порт и база).
2. Создать три базы в PostgreSQL и отдельного пользователя для каждой.
3. Собрать `auth-service`: перенести модель `User`, подписчик хэширования, контроллеры входа и профиля, `middleware` проверки токена и обработчик ошибок; добавить внутренние эндпоинты 1–3. Проверка: регистрация, вход и `/users/me` работают.
4. Собрать `restaurants-service`: перенести справочники, рестораны, администраторов, фото, меню, столики, слоты и отзывы; убрать внешние ключи на `user`; добавить HTTP-клиент к `auth` и внутренние эндпоинты 4–6. Проверка: каталог и поиск работают, отзывы показывают авторов.
5. Собрать `bookings-service`: перенести `Booking` (с новым полем `restaurant_id`), сервис бронирования и расчёт свободных столиков; убрать внешние ключи; добавить HTTP-клиент к `restaurants` и внутренний эндпоинт 7.

6. Подключить в `restaurants` проверку эндпоинтом 7 перед удалением столика, слота и ресторана.
7. Общий код (проверка JWT, обработчик ошибок, HTTP-клиент с таймаутом 3 с и проверкой внутреннего ключа) продублировать в каждый сервис.
8. Настроить `gateway` по таблице маршрутизации (на время разработки допустимо заменить его прямыми портами, полноценный `gateway` нужен для ЛР3–ЛР4).
9. Разделить тестовые данные: пользователи в `auth`, каталог в `restaurants`; идентификаторы пользователей в данных `restaurants` (администратор ресторана) должны совпадать с теми, что созданы в `auth`.
10. Прогнать коллекции Postman из ДЗ3 через `gateway` без изменений: критерий готовности состоит в том, что все 43 операции ведут себя как в монолите.

Выбран последовательный перенос архитектуры без работающего монолита вместо Strangler Fig: три сервиса собираются по очереди с проверкой, а `gateway` настраивается в конце.

Общее с Strangler Fig: перенос идёт по частям, а публичный API остаётся прежним. Но `gateway` появляется в конце, а не в начале, и монолит с новыми сервисами одновременно не работает.

Вывод

В ходе работы монолитное приложение для бронирования столиков разделено на три микросервиса (`auth`, `restaurants`, `bookings`) по принципу `database-per-service`: спроектированы границы сервисов, три независимые базы данных и архитектура с единой точкой входа (`gateway`). Найдены места, где части монолита связаны внешними ключами и общими запросами, и для каждого выбрано решение: логические ссылки, запросы по внутреннему API, вычисление свободных столиков в коде и проверка перед удалением вместо `ON DELETE RESTRICT`. Для межсервисного взаимодействия описано 7 внутренних эндпоинтов в формате OpenAPI с примерами запросов, ответов и ошибок, а также правила обработки сбоев. Публичный API из ДЗ2 остаётся неизменным, поэтому полученный план можно реализовать в ЛР2.